

PPC-Checkpoint-1

waypoints: coordinates of points along the track which defines the trajectory of the car's motion

↳ these are calculated from the cones & cam represent

- central line
- optimal path of the track (faster)
- safer path of the track

⊗ we deal with these discrete set of points & our task is to connect these waypoints to generate a smooth & feasible trajectory that the car can follow.

Interpolation: ~ to connect the discrete set of points & create a cont's path, there are diff methods for this & choosing the right one depends on the trade off b/w

- accuracy
- Smoothness
- Computation time
- Real-time feasibility.

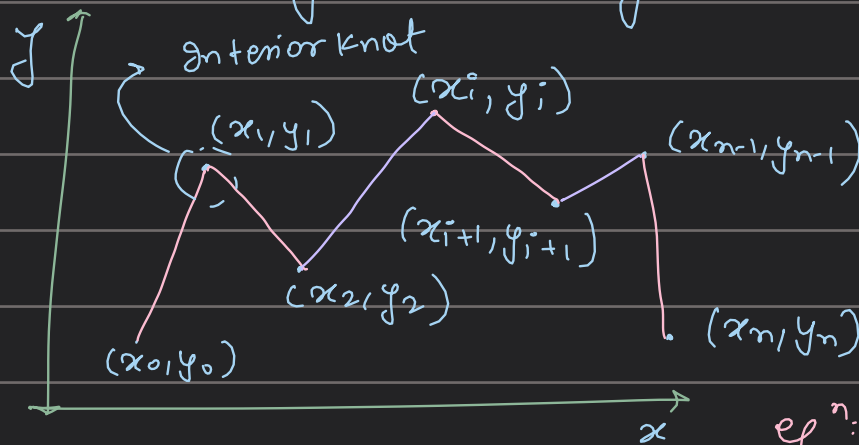
(A) Linear & Quadratic Spline Interpolation:

Linear Spline Interpolation:

Given $(x_0, y_0) \dots (x_n, y_n)$, interpolate the data to

Linear splines

↓
ascending or descending order: $x_0 < x_1 < \dots < x_n$



↓
connect the consecutive data points using st. lines

eqⁿ:

$$f(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} (x - x_i)$$

$$x_i \leq x \leq x_{i+1}$$

$$f_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0} (x - x_0) \quad x_0 \leq x \leq x_1$$

$$f_2(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1} (x - x_1) \quad x_1 \leq x \leq x_2$$

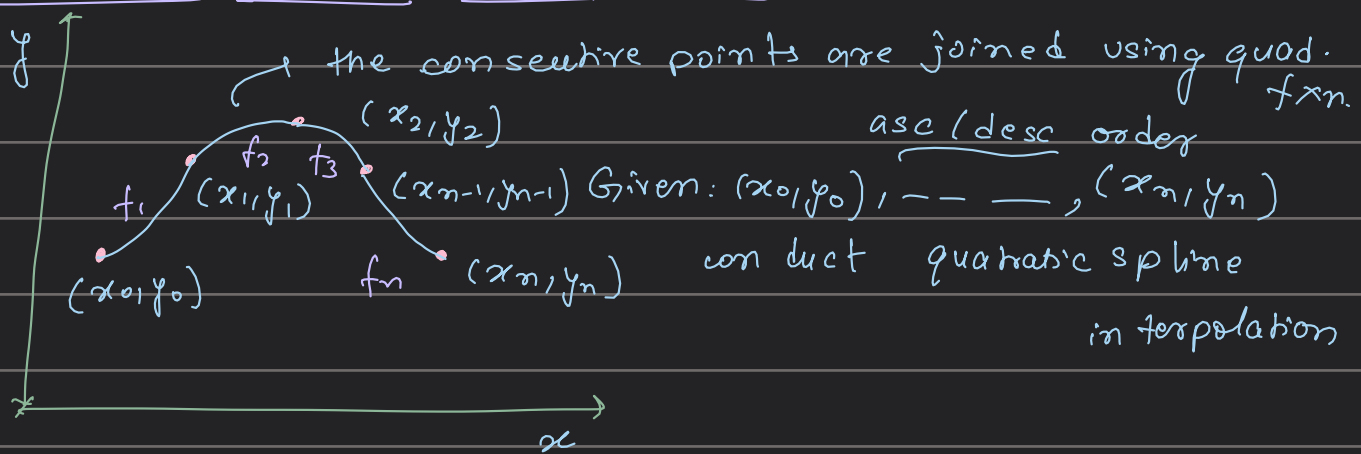
⋮

$$f_n(x) = y_{n-1} + \frac{y_n - y_{n-1}}{x_n - x_{n-1}} (x - x_{n-1}) \quad x_{n-1} \leq x \leq x_n$$

Limitations: • the value of y is not dependent on any other points other than the two points b/w.

• Interior knot: non diffble $\rightarrow$ sharp edge

Quadratic spline Interpolation:



$(n+1)$ data points

n splines

$3n$ unknowns

We req. $3n$ eqⁿ

$$f_1: a_1 x^2 + b_1 x + c_1$$

$$f_2: a_2 x^2 + b_2 x + c_2$$

⋮

$$f_n: a_n x^2 + b_n x + c_n$$

① each spline goes 2 consecutive data points $\rightarrow 2n$ eqⁿ ①

② continuity of polynomial at data points

$$f_i(x_{i-1}, y_{i-1}) = f_{i+1}(x_i, y_i)$$

$\rightarrow (n-1)$ eqⁿ ②

③ differentiability of polynomial at data points $\Rightarrow$ for smooth curve

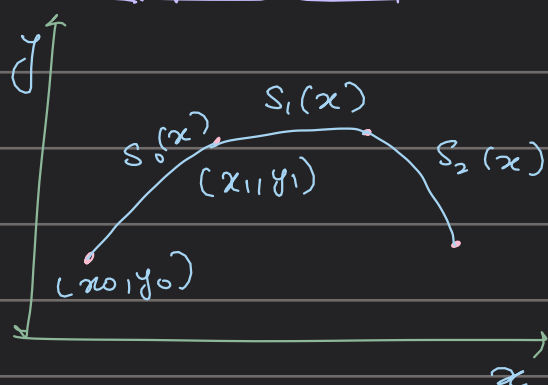
$(n-1) \text{ eqn} - (3)$

$f_i'(x_{i-1}, y_{i-1}) = f_i'(x_i, y_i)$

Using eqn ①, ②, ③, ④ we can find all the fn: $f_i, i=0$
 & let $a_1 = 0 - (4)$

(B) Cubic & Higher-Degree Spline Interpolation

Cubic Spline Interpolation



condⁿ at the data points

① $S_0(x_1) = S_1(x_1)$

② $S_0'(x_1) = S_1'(x_1)$

③ $S_0''(x_1) = S_1''(x_1)$

also, $S_0(x_0) = y_0$

$S_n(x_n) = y_n$

point satisfy the poly.

$\Rightarrow S_0''(x_0) = S_{n-1}''(x_n) = 0 \rightarrow$ assume

(Natural Boundary conditions)
(free)

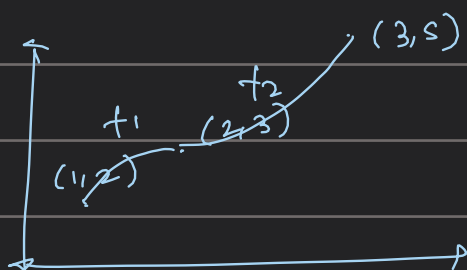
$\Rightarrow S_0'(x_0) = C_1, S_{n-1}'(x_n) = C_2 \rightarrow$ we should know the derivative at the ends points (clamped boundary)

Q) Construct a natural cubic spline that passes through the points (1,2), (2,3) & (3,5).

$f_1 = a_1 x^3 + b_1 x^2 + c_1 x + d_1$

$f_2 = a_2 x^3 + b_2 x^2 + c_2 x + d_2$

8 variables



$\Rightarrow a_1 + b_1 + c_1 + d_1 = 2 - (1)$

$8a_1 + 4b_1 + 2c_1 + d_1 = 3 - (2)$

$8a_2 + 4b_2 + 2c_2 + d_2 = 3 - (3)$

$27a_2 + 9b_2 + 3c_2 + d_2 = 5 - (4)$

$f_1' = 3a_1 x^2 + 2b_1 x + c_1$

$f_1'' = 6a_1 x + 2b_1$

$f_2' = 3a_2 x^2 + 2b_2 x + c_2$

$f_2'' = 6a_2 x + 2b_2$

$$\Rightarrow f_1'(2,3) = f_2'(2,3) \Rightarrow 12a_1 + 4b_1 + c_1 = 12a_2 + 4b_2 + c_2 \quad (5)$$

$$\Rightarrow f_1''(2,3) = f_2''(2,3) = 12a_1 + 2b_1 = 12a_2 + 2b_2$$

$$\Rightarrow 6a_1 + b_1 = 6a_2 + b_2 \quad (6)$$

$$\text{Since these are natural splines} \Rightarrow f_1''(1,2) = f_3''(3,5) = 0$$

$$\Rightarrow 3a_1 + b_1 = 0 \quad (7) \quad 9a_2 + b_2 = 0 \quad (8)$$

$$b_1 = -3a_1$$

$$b_2 = -9a_2$$

$$6a_1 - 3a_1 = 6a_2 - 9a_2 \Rightarrow 3a_1 = -3a_2 \Rightarrow a_1 = -a_2$$

$$12a_1 + (-12a_1) + c_1 = 12a_2 - 36a_2 + c_2$$

$$c_1 + 24a_2 = c_2$$

$$a_1 - 3a_1 + c_1 + d_1 = 2 \Rightarrow c_1 + d_1 = 2 + 2a_1$$

$$8a_1 - 12a_1 + 2c_1 + d_1 = 3 \Rightarrow 2c_1 + d_1 = 3 + 4a_1$$

$$c_1 = 1 + 2a_1$$

$$1 - 2a_1 + 24a_2 = c_2$$

$$c_1 = 1 - 2a_2$$

$$c_2 = 1 - 22a_2$$

$$8a_2 - 36a_2 + 2 - 44a_2 + d_2 = 3 \Rightarrow d_2 = 3 + 70a_2$$

$$a_2 = \frac{1}{15}$$

$$d_2 = 3 + \frac{14}{3} = \frac{23}{3}$$

$$c_2 = -\frac{7}{15}$$

$$b_2 = -\frac{3}{5}$$

$$a_1 = -\frac{1}{15}$$

$$b_1 = \frac{1}{5}$$

$$c_1 = 1 - \frac{2}{15}$$

$$d_1 = \frac{16}{15}$$

$$c_1 = \frac{13}{15}$$

$$f_1 = \frac{1}{15} (-x^3 + 3x^2 + 13x + 16)$$

$$f_2 = \frac{1}{15} (x^3 - 9x^2 - 7x + 23)$$

Cubic Natural
splines

Task-1 (01 - interpolation spline)

Commands + Code + Text ▶ Run all

RAM Disk

```
[5]
import matplotlib.pyplot as plt
import numpy as np
import math
```

```
x = np.array([1, 2, 3, 4])
y = np.array([1, 3, 2, 3])
```

$(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4)$

```
A = np.zeros((12, 12))
B = np.zeros(12)
```

12×12 coeff matrix

```
eqn = 0
```

$\rightarrow$ no. of eqn required : $(\text{no. of points} - 1) * 4 = 12$

```
for i in range(3):
    A[eqn, 4*i] = 1
    B[eqn] = y[i]
    eqn += 1
```

represents $S_i(x_i) = y_i$

```
dx = x[i+1] - x[i]
A[eqn, 4*i+1] = 1
A[eqn, 4*i+2] = dx**2
A[eqn, 4*i+3] = dx**3
B[eqn] = y[i+1]
eqn += 1
```

$S_{i+1}(x_{i+1}) = y_{i+1}$

```
for i in range(1, 3):
    dx = x[i] - x[i-1]
    A[eqn, 4*(i-1)+1] = 1
    A[eqn, 4*(i-1)+2] = 2*dx
    A[eqn, 4*(i-1)+3] = 3*dx**2
```

$S'_{i-1}(x_i) = S'_i(x_i)$

```
A[eqn, 4*i+1] = -1
eqn += 1
```

```
for i in range(1, 3):
    dx = x[i] - x[i-1]
    A[eqn, 4*(i-1)+2] = 2
    A[eqn, 4*(i-1)+3] = 6*dx
```

$S''_{i-1}(x_i) = S''_i(x_i)$

```
A[eqn, 4*i+2] = -2
eqn += 1
```

$\rightarrow$ to ensure

```
A[eqn, 2] = 2
eqn += 1
```

$S''_1(x_1) = 0$

```
dx = x[3] - x[2]
A[eqn, 10] = 2
A[eqn, 11] = dx*6
eqn += 1
```

$S_3''(x_4) = 0$

```
coeffs = np.linalg.solve(A, B)
```

solves $Ax = B$

Algorithm: Gaussian elimination

Output: all 12 coeff

```
def computePoly(i, xi):
    a = coeffs[4*i]
    b = coeffs[4*i+1]
    c = coeffs[4*i+2]
    d = coeffs[4*i+3]
```

extracting coeff from $\text{np.linalg.solve}(A, B)$

```
return a + b*(xi-x[i]) + c*(xi-x[i])**2 + d*(xi-x[i])**3
```

$\rightarrow$ finds all the 3 cubic polynomial

```
X, Y = [], []
```

```
for i in range(3):
    xi_vals = np.linspace(x[i], x[i+1], 50)
    yi_vals = computePoly(i, xi_vals)
    X.extend(xi_vals)
    Y.extend(yi_vals)
```

```
plt.plot(X, Y, label="Cubic Spline Interpolation")
plt.scatter(x, y, color='blue')
plt.legend()
plt.show()
```

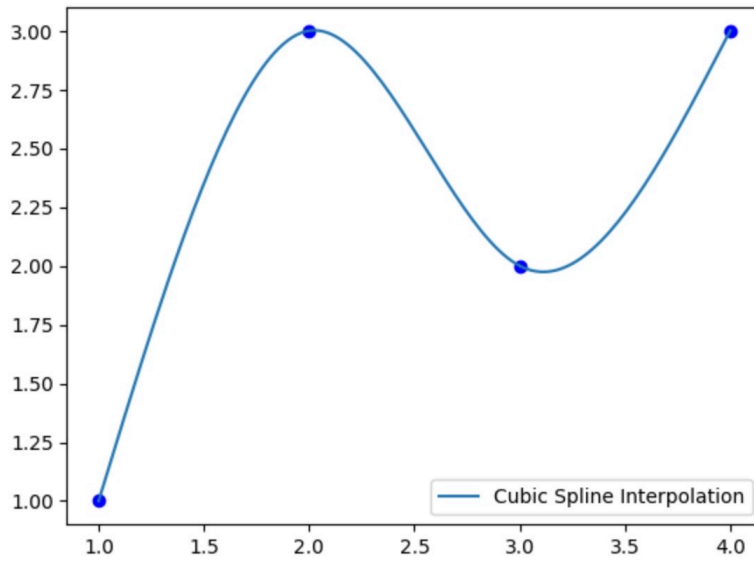
$Ax = B$ array
1D numpy array
2D numpy array

we have to find 3 cubic spline equations

$A[0,0] = A[0,4] = A[0,8] = 1$



...



[5]

✓ 0s

Start coding or [generate](#) with AI.



Task-2 (02-interpolation-spline-using-libraries)

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.interpolate import splprep, splev
from google.colab import files

uploaded = files.upload()
dataPoints = pd.read_csv('loop_track_waypoints.csv')

x = dataPoints['X'].values
y = dataPoints['Y'].values

if (x[0] != x[-1]) or (y[0] != y[-1]):
    x = np.append(x, x[0])
    y = np.append(y, y[0])

tck, u = splprep([x, y], s=0, per=True)

u_new = np.linspace(0, 1, 500)
x_smooth, y_smooth = splev(u_new, tck)

plt.figure(figsize=(6,6))
plt.plot(x_smooth, y_smooth, label='Cubic Spline', linewidth=2)
plt.plot(x, y, '--', label='Linear Interpolation')
plt.scatter(x, y, color='red', label='Waypoints')

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Cubic vs Linear Interpolation")
plt.legend()
plt.grid()
plt.axis('equal')

plt.show()

```

`splprep` → fits a parametric spline through a set of (x, y) points
`splev` → evaluates that spline at new parameter values to get smooth word.

reading & storing values of the row 'X' in x & row 'Y' in y from the csv file
 (arrays)

checking if first == last for loop closure
 if they are not equal then make them equal

creates the polynomial
 fits parametric cubic spline
 tck: spline definitions
 knots (where pieces join)
 coeff degree
 u: parameter values

⊗ `u_new = np.linspace`: create a set from 0 to 1 containing 500 equally spaced numbers

for cubic
`x_smooth, y_smooth = splev(u_values, tck)` to generate the smooth curve

takes each value of u & uses spline(tck) to compute $x(u), y(u)$
`x_smooth` → 500 x-coords
`y_smooth` → 500 y-coords

it can create any splines but
`tck, u = splprep([x, y], s=0, per=True)` by default $k=3$ is set

...

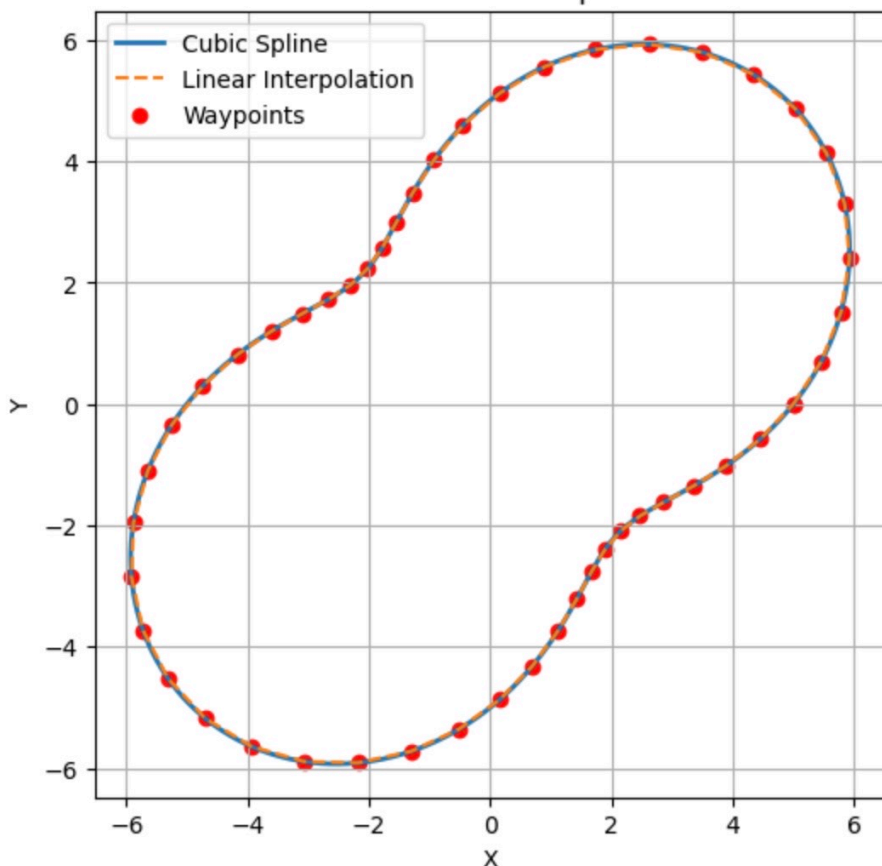
Choose Files

loop_track_waypoints.csv

loop_track_waypoints.csv(text/csv) - 1982 bytes, last modified: n/a - 100% done

Saving loop_track_waypoints.csv to loop_track_waypoints (3).csv

Cubic vs Linear Interpolation



Fitting the spline:

$tck, u = \text{splprep}([x, y], s=0, \text{per}=\text{True})$

function

passes both
coord arrays
together as a
2D parametric
curve

per: periodic
curve (here closed loop)

* when s inc the curve becomes smoother but less accurate

$tck \rightarrow$ a tuple containing the knots, coeff, degree of the spline
(internal representation used by SciPy)

$u \rightarrow$ the parameter values corresponding to each original waypoint (0-1)

normalised "progress along the track", $u=0$ start, $u=1$ end

to track progress along the track to see how many waypoints are taken

this tck, u

into consideration so far

↳ this is the core step to get the parametric curve

$u_new = np.linspace(0, 1, 500)$

making the values of

$x_smooth, y_smooth = splev(u_new, tck)$

parameter u such that
the spline becomes

↳ this is done to convert
the spline from linear (x, y) to cubic (x_smooth, y_smooth)
smoother

$tck, u = splprep([x, y], s=0, per=True)$

↳ $splprep$ by default fits a curve of degree = 3

↳ By using $splev$ we got new points x_smooth & y_smooth , plotting which will give a cubic spline

$u_new = np.linspace(0, 1, 500)$

↳ this number changes the smoothness

of the curve & not the spline

points → connect → curve

(S)

working of $splprep$: takes $[x, y] \rightarrow x = f(u), y = g(u)$

↳ builds a parametric spline

output: $tck = (t, c, K)$

↓ ↓ degree
knots coeff

fully defines
the spline

working of $splev$: it evaluates the spline mathematically

input: u_new & tck , it returns x_new & y_new
↓
spline definition

⊗ $splprep$: builds the eqⁿ, $splev$: plugs values into the eqⁿ

